

轻量级分组密码算法 FBC^{*}

冯秀涛¹, 曾祥勇², 张 凡³, 曾 光⁴, 唐 灯⁵, 甘国华^{6,7}, 王永兴^{1,8}

1. 中国科学院 数学与系统科学研究院, 中国科学院数学机械化重点实验室, 北京 100190
2. 湖北大学 数学与统计学学院, 应用数学湖北省重点实验室, 武汉 430062
3. 兴唐通信科技有限公司, 北京 100191
4. 战略支援部队信息工程大学 网络空间安全学院 数学工程与先进计算国家重点实验室, 郑州 450001
5. 西南交通大学 数学学院, 成都 610031
6. 北京太一云科技有限公司, 北京 100125
7. 北京科技大学 计算机与通信工程学院, 北京 100083
8. 中国科学院大学 数学学院, 北京 100049

通信作者: 冯秀涛, E-mail: fengxt@amss.ac.cn

摘 要: FBC 是一族轻量级分组密码算法, 主要包含 FBC128-128, FBC128-256 和 FBC256-256 三个版本, 可支持 128 和 256 两种比特长度的明文分组以及 128 和 256 两种比特长度的密钥. FBC 算法采用 4 路两重 Feistel 结构设计, 在结构上通过增加两个异或操作的微小代价较大提高整体结构的扩散特性. 非线性函数 F 采用切片技术, 其中, S 盒基于 NFSR 构造, 其各项密码学性质达到最优, 同时硬件实现代价达到最小, 为最轻的 S 盒之一; 线性变换 L 仅由循环移位和异或构成, 具有较好的密码学特性的同时兼顾好的软硬件实现效能. 在安全性方面, 我们基于混合整数线性规划 (MILP) 寻找最少活跃 S 盒个数的方法对其进行差分、线性、不可能差分以及积分等方法的分析. 实验结果表明, FBC 可以抵抗上述各种攻击方法的攻击. FBC 算法不仅具有结构简洁, 轻量化, 安全性高等特性, 还具有灵活高效的软硬件实现方式, 可以满足不同平台的应用需求.

关键词: 轻量级分组密码; Feistel 结构; FBC 算法; 混合整数线性规划; 比特切片

中图分类号: TP309.7 **文献标识码:** A **DOI:** 10.13868/j.cnki.jcr.000340

中文引用格式: 冯秀涛, 曾祥勇, 张凡, 曾光, 唐灯, 甘国华, 王永兴. 轻量级分组密码算法 FBC[J]. 密码学报, 2019, 6(6): 768–785.

英文引用格式: FENG X T, ZENG X Y, ZHANG F, ZENG G, TANG D, GAN G H, WANG Y X. On the lightweight block cipher FBC[J]. Journal of Cryptologic Research, 2019, 6(6): 768–785.

On the Lightweight Block Cipher FBC

FENG Xiu-Tao¹, ZENG Xiang-Yong², ZHANG Fan³, ZENG Guang⁴, TANG Deng⁵, GAN Guo-Hua^{6,7}, WANG Yong-Xing^{1,8}

* 基金项目: 国家自然科学基金 (61572491, 61761166010, 61872435); 国家重点研发计划 (2016YFB0800401, 2017YFB0803203); 数学工程与先进计算国家重点实验室开放基金 (2018A06); 太一云项目 (Y790101)

Foundation: National Natural Science Foundation of China (61572491, 61761166010, 61872435); National Key Research and Development Program of China (2016YFB0800401, 2017YFB0803203); Open Fund of State Key Laboratory of Mathematical Engineering and Advanced Computing (2018A06); Tai Cloud Project (Y790101)

收稿日期: 2019-11-30 定稿日期: 2019-12-10

1. Key Laboratory of Mathematics Mechanization, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing 100190, China
2. Hubei Key Laboratory of Applied Mathematics, Faculty of Mathematics and Statistics, Hubei University, Wuhan 430062, China
3. Xingtang Communication Technology Co. Ltd., Beijing 100191, China
4. School of Mathematics, Southwest Jiaotong University, Chengdu 610031, China
5. State Key Laboratory of Mathematical Engineering and Advanced Computing, School of Cyberspace Security, PLA Strategic Support Force Information Engineering University, Zhengzhou 450001, China
6. Tai Cloud Technology LTD, Beijing 100125, China
7. School of Computer and Communication Engineering, University of Science and Technology Beijing, Beijing 100083, China
8. School of Mathematics Sciences, University of Chinese Academy of Sciences, Beijing 100049, China

Corresponding author: FENG Xiu-Tao, E-mail: fengxt@amss.ac.cn

Abstract: FBC is a family of lightweight block cipher algorithms, which has three versions: FBC128-128 with block size and key size being both 128 bits, FBC128-256 with block size being 128 bits and key size being 256 bits, and FBC256-256 with block size and key size being both 256 bits. FBC is based on a 4-branch two-fold Feistel structure whose diffusion property is enhanced greatly by means of increasing the cost of two XOR operations. The nonlinear function F adopts the bit-slice technology. The S-box S involved in F is based on a nonlinear feedback shift register (NFSR), which possesses a number of optimal cryptographic properties including minimal differential uniform, maximal nonlinearity, no fixed points, etc., and is one of the lightest S-boxes. The linear transformation L consists only of rotation and XOR operators, hence it has good cryptographic properties and high performance in software and hardware implementations. As for FBC's overall security, the mixed integer linear programming (MILP) method was used to find the minimal number of active S-boxes during differential analysis, linear analysis, impossible differential analysis, and integral analysis. Experimental results show that FBC can resist the above attacks. FBC has the characteristics of simple structure, is lightweight, has high security, and is flexible and efficient in software and hardware implementations.

Key words: lightweight block cipher; Feistel structure; FBC; MILP; bit-slice

1 概述

分组密码是一种典型的对称密码算法, 它采用固定的密码函数, 在密钥的控制下对输入的明文消息进行加密得到密文消息, 或者对密文消息进行解密得到明文消息. 典型的分组密码算法包括美国早期的数据加密标准 DES、现行的高级加密标准 AES、我国的商业密码行业标准 SM4^[1] 等. 分组密码结合密码模式已被广泛用在安全存储、网络通信加密中.

轻量级分组密码是一种特殊的分组密码体制, 其主要是指密码算法硬件实现和运行时所占用的系统资源的开销较少, 例如硬件实现面积、运行时所占内存空间大小和能耗等. 这类分组密码算法主要被用于资源受限的计算环境中, 例如电子标签 RFID. 随着物联网的快速发展, 物联网终端传感器部件广泛部署, 对轻量级密码体制的需求也越来越大. 由于轻量级分组密码具有较低的实现成本和较低的能耗, 在物联网终端、卫星通信网络中大量部署轻量级密码算法可以带来显著的社会经济效益.

分组密码的典型结构包括 Feistel 结构和 SPN 结构等. 这两种结构已经得到广泛深入的研究^[2,3]. 切片 (bit-slice) 整体上属于 SPN 结构, 最早由 Eli Biham 于 1997 年提出^[4], 是一种构建轻量级分组密码的重要技术之一. 当前 Present^[5]、SERPENT^[6]、Rectangle^[7]、Keccak^[8] 等密码均采用了这一技术来设计算法.

安全是密码算法得以应用的前提. 近年来, 分组密码的安全性评估已取得长足的进步. 特别是, 基于混合整数线性规划 (MILP) 方法^[9-11]的提出, 使得分组密码算法安全分析朝着自动化、精细化、标准化、可证明方向发展.

本文主要描述了一种轻量级的分组密码算法 FBC. 该算法主要基于 Feistel 结构设计, 并以 Feistel-based Block Cipher 的首字母命名. FBC 算法主要包括 FBC128-128, FBC128-256 和 FBC256-256 三个版本, 分别支持 128 和 256 两种比特长度的明文分组以及 128 和 256 两种比特长度的密钥. 在整体结构上, FBC 算法采用 4 路两重 Feistel 结构设计. 在保持结构简洁的同时我们通过增加两个异或操作的微小代价较大提高 FBC 算法整体结构的扩散特性. 非线性函数 F 采用切片技术. 其中, S 盒基于 NFSR 构造, 其各项密码学性质达到最优, 同时硬件实现代价达到最小, 为最轻的 S 盒之一; 线性变换 L 仅由循环移位和异或构成, 具有较好的密码学特性的同时兼顾好的软硬件实现效能. 在安全性方面, 我们基于混合整数线性规划 (MILP) 寻找最少活跃 S 盒个数的方法对其进行差分、线性、不可能差分以及积分等方法的分析. 实验结果表明, FBC 可以抵抗上述各种攻击方法的攻击. FBC 算法不仅具有结构简洁, 轻量化, 安全性高等特性, 还具有灵活高效的软硬件实现方式, 可以满足不同平台的应用需求.

本文余下章节安排如下: 第 2 节详细描述了 FBC 算法规范流程; 第 3 节给出了 FBC 算法设计准则; 第 4 节给出了 FBC 算法部件的安全属性, 并在第 5 节给出了算法的整体安全评估结果; 最后在第 6 节对算法的软硬件实现性能作了简单评估.

2 算法规范

本节详细描述了轻量级分组密码 FBC 的算法流程, 包括密钥扩展算法、加密过程和解密过程. 其不同版本以格式 $\text{FBC}n\text{-}m$ 统一进行命名, 其中 n 为明文分组比特长度, m 为主密钥比特长度.

2.1 算法参数

FBC 不同版本的状态均由 4 个 w 比特的字组成, 其中字长 w 与明文分组长度 n 的关系满足 $w = n/4$. 设 r 为迭代轮数. FBC 算法主要参数见表 1.

表 1 FBC 主要参数
Table 1 Main parameters of FBC

版本	分组长度 n	主密钥长度 m	字宽度 w	迭代轮数 r
FBC128-128	128	128	32	48
FBC128-256	128	256	32	64
FBC256-256	256	256	64	80

2.2 符号

本文主要使用下面的一些符号:

- P : n 比特明文
- C : n 比特密文
- k_i : 第 i 个子密钥, $i = 0, 1, \dots, 2r + 1$
- a_i, b_i, c_i, d_i : 第 i 轮迭代的状态字, 每个字的长度为 w 比特, $i = 0, 1, \dots, r + 1$
- u, v : 长度为 w 比特的字
- u_i, v_i : 字 u 和 v 的第 i 个长度为 $w/4$ 比特的子块, $i = 1, 2, 3, 4$
- $u_i[j], v_i[j]$: 子块 u_i 和 v_i 的第 j 比特, $j = 0, 1, 2, \dots, w/4 - 1$
- $\mathbf{1}$: 长度为 w 比特的全 1 子块

和运算符:

- $\oplus$: 面向比特的异或运算
- $\&$: 面向比特的与运算
- $\lll$: w 比特的字左循环移位运算
- $\parallel$: 字符串连接符

2.3 算法描述

2.3.1 算法结构

FBC 算法基于 4 路两重 Feistel 结构设计, 非线性函数 F 采用切片技术构造, 算法主体结构如图 1.

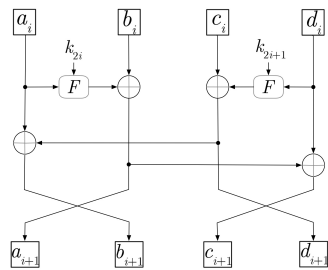


图 1 FBC 算法结构图
Figure 1 Structure of FBC

2.3.2 非线性函数 F

非线性函数 F 的输入包含 2 个 w 比特的字 x 和 y , 输出一个 w 比特的字 z , 记 $z = F(x, y)$. 具体运算过程如下.

- (1) 子密钥加: $u = x \oplus y$.
- (2) 列变换: 将 u 二进制表示分解成 4 个等宽的字符串, 每个字符串包含 $w/4$ 个比特, 即 $u = u_1 \parallel u_2 \parallel u_3 \parallel u_4$, 令:

$$v_1[j] \parallel v_2[j] \parallel v_3[j] \parallel v_4[j] = S(u_1[j] \parallel u_2[j] \parallel u_3[j] \parallel u_4[j]), j = 0, 1, \dots, w/4 - 1$$

这里 S 为 4 进 4 出的 S 盒变换, 见表 2. 记 $v = v_1 \parallel v_2 \parallel v_3 \parallel v_4$.

- (3) 行变换 L :

$$z = L_{s,t}(v) = v \oplus (v \lll s) \oplus (v \lll t)$$

在行变换 $L_{s,t}(v)$ 中参数 s, t 取值与 w 有关, 见表 3.

表 2 S 盒真值表
Table 2 Truth table of S-box

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$S(x)$	5	10	15	4	9	14	11	8	2	7	12	13	3	6	1	0

表 3 L 线性变换中循环参数 s, t 取值
Table 3 Value of s, t in linear transformation

w	32	64
s	3	17
t	10	58

2.3.3 密钥扩展算法 ExpandKey

针对 FBC128-128 和 FBC256-256, 将 m 比特主密钥 k 分成 4 个 w 比特的字 k_0, k_1, k_2, k_3 , 即 $k = k_0 \parallel k_1 \parallel k_2 \parallel k_3$. 对 $i = 0, 1, \dots, 2r - 5$, 计算

$$k_{i+4} = (k_i \oplus 1) \oplus (k_{i+1} \oplus 1) \& k_{i+2} \& k_{i+3} \oplus i$$
$$k_{i+4} = L_{7,25}(k_{i+4}) = k_{i+4} \oplus (k_{i+4} \lll 13) \oplus (k_{i+4} \lll 22)$$

针对 FBC128-256, 将 256 比特主密钥 k 分成 8 个 32 比特的字 $k_0, k_1, k_2, \dots, k_7$, 即 $k = k_0 \parallel k_1 \parallel \dots \parallel k_7$. 对 $i = 0, 1, \dots, 2r - 9$, 计算

$$\begin{aligned} k_{i+8} &= (k_i \oplus 1) \oplus (k_{i+1} \oplus 1) \& k_{i+2} \& k_{i+3} \oplus i \\ k_{i+8} &= L_{7,25}(k_{i+8}) = k_{i+8} \oplus (k_{i+8} \lll 13) \oplus (k_{i+8} \lll 22) \end{aligned}$$

2.3.4 加密过程

- (1) 首先根据 2.3.3 节密钥扩展算法将主密钥 k 扩展成子密钥序列 $k_i, i = 0, 1, \dots, 2r - 1$;
- (2) 将输入明文块 P 分成 4 个长度为 w 比特的子块, 即 $P = a_0 \parallel b_0 \parallel c_0 \parallel d_0$. 重复执行下列操作 $r - 1$ 次 (RoundFunc):

- (a) $a_{i+1} = F(a_i, k_{2i}) \oplus b_i$
- (b) $c_{i+1} = a_{i+1} \oplus d_i$
- (c) $d_{i+1} = F(d_i, k_{2i+1}) \oplus c_i$
- (d) $b_{i+1} = d_{i+1} \oplus a_i$

- (3) 然后执行下列操作 1 次 (FinalRoundFunc)(不交换顺序):

- (a) $b_{r+1} = F(a_r, k_{2r-2}) \oplus b_r$
- (b) $d_{r+1} = b_{r+1} \oplus d_r$
- (c) $c_{r+1} = F(d_r, k_{2r-1}) \oplus c_r$
- (d) $a_{r+1} = c_{r+1} \oplus a_r$

- (4) 最后输出密文块 $C = a_{r+1} \parallel b_{r+1} \parallel c_{r+1} \parallel d_{r+1}$.

2.3.5 解密过程

- (1) 首先根据第 2.3.3 节密钥扩展算法将主密钥 k 扩展成子密钥序列 $k_i, i = 0, 1, \dots, 2r - 1$;
- (2) 将输入密文块 C 分成 4 个长度为 w 比特的子块, 即 $C = a_0 \parallel b_0 \parallel c_0 \parallel d_0$. 首先执行下列操作 1 次:

- (a) $a_1 = a_0 \oplus c_0$
- (b) $b_1 = F(a_1, k_{2r-2}) \oplus b_0$
- (c) $d_1 = b_0 \oplus d_0$
- (d) $c_1 = F(d_1, k_{2r-1}) \oplus c_0$

- (3) 然后对 $i = 1, 2, \dots, r$, 重复执行下列操作 $r - 1$ 次:

- (a) $a_{i+1} = b_i \oplus d_i$
- (b) $b_{i+1} = F(a_{i+1}, k_{2(r-i)-2}) \oplus a_i$
- (c) $d_{i+1} = a_i \oplus c_i$
- (d) $c_{i+1} = F(d_{i+1}, k_{2(r-i)-1}) \oplus d_i$

- (4) 最后输出明文块 $P = a_{r+1} \parallel b_{r+1} \parallel c_{r+1} \parallel d_{r+1}$.

3 设计准则

本节给出了设计 FBC 算法时的一些参考准则, 包括整体设计准则和部件设计准则两大类.

3.1 整体设计准则

FBC 算法整体上遵循简单、简洁、轻量、高效、安全等准则设计. 其主要体现于:

- **简单:** FBC 算法基础运算部件简单, 仅仅由通用部件异或、循环移位、逻辑与和 S 盒等构造. 整个加解密过程和密钥扩展过程均简单明了.
- **简洁:** FBC 算法结构简洁, 整体采用 4 路两重 Feistel 结构设计, F 函数基于切片技术构造, 整个算法没有任何冗余操作.
- **轻量:** FBC 算法部件采用轻量化设计, 其中 S 盒基于 NFSR 构造, 硬件实现仅需 5 个非门、4 个或门和 4 个与门. 线性变换 L 每个输出比特仅需 2 个或门. 密钥扩展同样采用轻量化设计, 每轮子密钥比特硬件实现仅需 2 个非门, 4 个异或门和 2 个与门.
- **高效:** FBC 算法具有多种高效的软硬件实现方法. 在软件实现方面, 可以采用 4 进 w 出或者 8 进 w 出表实现, 亦可采用无查表快速逻辑实现方法进行实现; 在硬件实现方面, 局部可以基于 NFSR 对 S 盒和密钥扩展高效实现, 整体既可以同时实现左右两路 Feistel 结构也可以只实现一侧 Feistel 结构.
- **安全:** FBC 算法整体采用 4 路两重 Feistel 结构, F 函数采用切片技术构造, 局部属于 SPN 结构, 其中 S 盒选择的是密码学性质最优的 S 盒, 置换 P 选择差分/线性分支数为 4 的线性变换 L , 在结构上没有弱点. 只要迭代足够的轮数, 整个算法就具有较高的安全性.

3.2 部件设计准则

3.2.1 S 盒

S 盒在设计时主要遵循了如下准则:

- 4 进 4 出的置换
- 具有最优的差分均匀度和非线性度
- 至少有一个分量布尔函数代数次数为 3
- 没有不动点
- 轻量化
- 具有高效的软件实现
- 在上述条件下硬件实现代价最低

根据上述准则, 我们选择基于 4 级 NFSR 来构造最优的 4 进 4 出 S 盒. 其中 NFSR 反馈函数 f 选择如下:

$$f(x_0, x_1, x_2, x_3) = 1 \oplus x_0 \oplus x_3 \oplus x_2 x_3$$

设由 f 定义的李FSR 生成的序列的前 8 项为 $x_0 x_1 x_2 x_3 x_4 x_5 x_6 x_7$. 令 $x = x_0 x_1 x_2 x_3$ 和 $y = x_4 x_5 x_6 x_7$. 则 NFSR 诱导出 F_2^4 到 F_2^4 上的一个置换 $S: y = S(x)$.

3.2.2 线性变换 L

线性变换 L 在设计时主要遵循了如下准则:

- w 进 w 出的线性置换
- 差分分支数和线性分支数不低于 4
- 具有最高的周期
- 具有最少个数的不动点
- 具有最优的全扩散特性
- 简单, 便于软硬件实现
- 轻量化

根据上述准则, 我们选择如下仅由异或和循环移位两种运算构成的线性变换:

$$L_{s,t}(v) = v \oplus (v \lll s) \oplus (v \lll t)$$

其中参数 s, t 取值与 w 有关, 见表 3.

3.2.3 密钥扩展

密钥扩展在设计时主要遵循了如下准则:

- 非线性置换
- 简单
- 轻量化
- 高效软硬件实现
- 能抵抗滑动攻击

根据上述准则, 我们采用类似切片技术设计了密钥扩展, 在纵向列变换上采用 M 序列构造方法, 在横向行变换上面采用线性变换 L , 在满足轻量化需求下达到快速混淆和扩散的目的.

4 部件分析

4.1 S 盒

定义 1 (代数次数) 设正整数 n . 一个 $n \times n$ 的 S 盒是 F_2^n 到 F_2^n 上的一个映射: $s = (f_1, f_2, \dots, f_n)$, 这里 f_i ($1 \leq i \leq n$) 均是 n 元布尔函数. S 盒 s 的代数次数为其分量函数的线性组合的最低代数次数, 即:

$$\deg(s) = \min\{\deg(c_1 f_1 + c_2 f_2 + \dots + c_n f_n) | 0 \neq (c_1, c_2, \dots, c_n) \in F_2^n\}$$

定义 2 (非线性度) 对于一个 $n \times n$ 的 S 盒 $y = s(x)$, 定义 S 盒 s 的非线性度为

$$NL(s) = 2^{n-1} - \frac{1}{2} \max_{v \neq 0, v \in F_2^n, w \in F_2^n} \left| \sum_{x \in F_2^n} (-1)^{\langle v, F(x) \rangle + \langle w, x \rangle} \right|$$

其中 $\langle, \rangle$ 表示内积运算.

定义 3 (差分均匀度) 对于一个 $n \times n$ 的 S 盒 $y = s(x)$, 定义

$$\max_{\Delta x \neq 0, \Delta x, \Delta y \in F_2^n} |\{x | S(x + \Delta x) + S(x) = \Delta y\}|$$

为该 S 盒 s 的差分均匀度.

定义 4 (图代数免疫度) 对于一个 $n \times n$ 的 S 盒 $y = s(x)$, 定义

$$\min\{\deg(F) | 0 \neq F \in B_{2n}, F(x, y) = 0, \forall x \in F_2^n, y = S(x)\}$$

为该 S 盒 s 的图代数免疫度.

定义 5 (不动点) 对于一个 $n \times n$ 的 S 盒 $y = s(x)$, 称满足 $s(x) = x$ 的解为 S 盒的不动点.

对于 FBC 算法选择的 4 进 4 出的 S 盒的密码性质见表 4, 其差分均匀性、非线性度、图代数免疫度都达到了最优, 没有不动点.

表 4 S 盒密码学指标
Table 4 Cryptographic properties of S-box

差分均匀度	非线性度	图代数免疫度	代数次数	不动点个数
4	4	2	2	0

4.2 线性变换 L

定义 6 (线性变换) 有限域 F_q 上的一个变换 L 被称之为是 F_q 上的 n 维线性变换, 是指其对任意 $X, Y \in F_q^n, c \in F_q$, 均满足

$$L(X + Y) = L(X) + L(Y), L(cX) = cL(X)$$

定义 7 (差分分支数) 对一个线性变换 $L: F_q^n \rightarrow F_q^n, Y = L(X)$, 可以用矩阵的形式表示, 即存在 F_q 上的一个 $n \times n$ 的矩阵 M , 使得对任意 X , 有 $Y = L(X) = MX$. 线性变换 L 的差分分支数定义为

$$B_D = \min_{x \neq 0} (w_H(X) + w_H(MX))$$

其中 $w_H(X)$ 是 X 的汉明重量, 也就是非零分量的个数.

由定义可知, 一个线性变换 L 的差分分支数总是小于等于 $n + 1$.

定义 8 (线性分支数) 对线性变换有 $Y = L(X) = MX$, 其线性分支数定义为

$$B_L = \min_{0 \neq \alpha \in F_q^n} (w_H(\alpha) + w_H(M'\alpha))$$

这里 M' 表示矩阵 M 的转置.

定义 9 (周期) 对任意给定的非零变换 P 和正整数 i , 定义

$$P^{(i)}(x) = P(P^{(i-1)}(x)), P^{(1)}(x) = P(x), P^{(0)}(x) = x$$

称使得对所有 x 均有 $P^{(i)}(x) = x$ 成立的最小正整数 i 为变换 P 的周期.

对于 FBC 算法族选择的线性变换 $L_{s,t}$, 其密码性质如下:

表 5 线性变换 $L_{s,t}$ 的密码学指标
Table 5 Cryptographic properties of $L_{s,t}$

差分分支数	线性分支数	周期	不动点个数
4	4	w	2

5 安全性评估

5.1 差分分析

差分分析是最初由 Biham E 和 Shamir A 于 1991 年针对 DES 算法提出的一种密码分析方法^[12]. 该方法几乎适用于所有的分组密码体制, 是对分组密码算法威胁最大的分析方法之一^[13-15]. 对于采用 S 盒作为核心部件的分组密码算法, 评估单条差分传播路径最大概率上界的最有效方法就是估算差分传播路径中活动 S 盒的个数. 2011 年 Mouha N 等人^[9] 利用混合整数线性规划 MILP 给出差分分析和线性分析中活动 S 盒个数下界的评估方法. 他们首先将差分分析和线性分析中活动 S 盒传播规律用线性不等式刻画, 目标函数为最小的活动 S 盒个数, 然后利用求解 MILP 问题的软件进行求解, 如 Cplex、Gurobi 等. 该方法得到的下界精度较高, 许多时候达到最优值, 因此被广泛用来作为评估分组密码抵抗差分分析和线性分析能力的参考指标.

对 FBC 算法, 我们采用 MILP 方法搜索了差分分析中活动 S 盒的最少个数. 由于采用原始方法利用线性不等式直接刻画 S 盒 s 和线性变换 L 以及异或 $\oplus$ 等运算涉及的变元个数和不等式个数均较多, 运行 Gurobi 等软件时能够求解的轮数太少, 时间太长, 效率太低. 故我们采用一种简化的办法来求解活动 S 盒的个数. 该方法可以求解 FBC 算法全轮迭代过程中活动 S 盒的个数, 具体刻画过程见附录 A. 需要说明

的是,该方法得到的下界并不一定是真实差分分析中的活动 S 盒个数的精确下界,有时甚至相差很大.因此其结果仅仅作为一种参考.

作为结果,对 FBC128-128、FBC256-256 达到安全界的迭代轮数为 32 和 64,而实际算法迭代轮数为 48 和 80,我们认为它们完全可以抵抗差分密码分析;对 FBC128-256,迭代 64 轮时活动 S 盒个数为 128,刚好达到安全界.如果考虑到分组密码剥离轮技术的应用,通过该方法估算的活动 S 盒个数会低于 64.正如上面所说,一方面我们的 MILP 方法估算活动 S 盒个数结果很粗糙,虽然它是一个下界,但是离精确的下界可能相差很大,另外一个方面,在实际差分密码分析中,真实差分路径的概率往往比确切活动 S 盒个数估算的界往往小非常多,因此我们依然相信 FBC128-256 对于差分分析来说是安全的.

5.2 线性分析

线性分析最初由 Matsui M 于 1993 年针对 DES 算法提出的一种密码分析方法^[16].与差分分析相似,该方法同样适用于所有的分组密码体制,是对分组密码算法威胁最大的分析方法之一^[17-19].对于采用 S 盒作为核心部件的分组密码算法,评估单条线性迹(即线性掩码传播路径)的最大偏差概率的上界的最有效的方法就是估算线性迹中活动 S 盒的个数.类似于差分分析,我们同样采用简化的 MILP 方法对其线性迹中活动 S 盒的传播规律作了刻画,其可以求解 FBC 全轮迭代过程中活动 S 盒的个数,具体刻画过程见附录 B.

作为结果,对 FBC128-128、FBC256-256 达到安全界的迭代轮数为 32 和 64,而实际算法迭代轮数为 48 和 80,我们认为它们完全可以抵抗线性密码分析;对 FBC128-256,迭代 64 轮时活动 S 盒个数为 128,刚好达到安全界.同样的道理,我们相信 FBC128-256 对于线性分析来说也是安全的.

5.3 不可能差分分析

不可能差分是差分分析的一种变体^[20-22],也是一种常见的密码分析方法.对于不可能差分分析,我们同样利用 MILP 方法来搜索 FBC 最长轮数的不可能差分模式,具体的转化 MILP 不等式方法与差分分析类似,不再赘述.我们在具体分析中构建了三种 MILP 模型,在有限的计算资源下,可得到 FBC128-128 和 FBC128-256 的 7 轮不可能差分 and FBC256-256 的 13 轮不可能差分.下面给出三种模型的构造方法,由已知的不可能差分分析结果来看,几乎所有的不可能差分路径的输入和输出差分模式分别只有一个活跃 S 盒,因此,三种模型中,选定输入输出差分时,应满足该输入(输出)只能使单个 S 盒活跃.

- **M1:** 变量基于比特,将第一轮输入差分和最后一轮的输出差分信息 (α, β) 添加到搜索 i (i 从第一轮开始遍历) 轮差分路径的模型中,去掉目标函数,判断该模型有无解即可.若 i 轮模型无解,说明存在 i 轮的不可能差分路径 (α, β) ,继续搜索 $i+1$ 轮;若 $i+1$ 轮有解,不再遍历 $i+1$ 轮之后的模型,此时不可能差分路径 (α, β) 轮数最长为 i .遍历输入输出差分 (α, β) ,重复上面过程,则可得到最大轮数的不可能差分和对应的差分选取模式.
- **M2:** 用到 Sasaki 和 Todo 提出的方法^[23],同 M1 类似,不同在于对 S 盒的刻画.这里对于 S 盒来说,任意的非 0 输入差分经过 S 盒都可以得到任意的非 0 输出差分,因此 S 盒的刻画只需要两个不等式,而且遍历差分模式时,只需要对活跃 S 盒的位置进行遍历, S 盒的输入输出任意取值即可.搜索过程与 M1 相同.
- **M3:** 将 S 盒作为变量,则只需刻画轮函数的线性层即可,相对于上述两种模型,变量减少大概 4 倍.遍历输入输出差分模式时,遍历活跃 S 盒的位置即可,搜索过程同上.

上述三种方法各有优劣. M1 遍历所有的输入输出差分,即使规定活跃 S 盒个数为 1,要搜索的模型个数也是远远大于其他两种,因此搜索时间最长;当轮数较大时, M3 能快速给出模型是否有解,效率上 $M3 > M2 > M1$;但由于 M1 遍历所有输入输出差分, M2 取定 S 盒的某个输入输出, M3 只是遍历 S 盒位置,因此在轮数大小上 $M1 > M2 > M3$.

在 FBC128 实验中,对于 M1 和 M2,由于计算能力有限,第八轮就已经不能计算出结果,第七轮无解; M3 虽然可判断的轮数大,但是 M3 在第七轮之后的所有轮数都有解.三个模型给出的结论一样,目前只能搜索到 7 轮不可能差分.

表 6 FBC 算法不可能差分分析结果
Table 6 Impossible differential analysis of FBC

	M1	M2	M3
FBC128-128 和 FBC128-256	目前搜出 7 轮, 8 轮单个模型搜索时间超过 34h, 无结果无法判断	同 M1	最长不可能差分 7 轮, 超过 7 轮的所有模型均有解
FBC256-256	同 M2	目前搜索到最长不可能差分 13 轮, 14 轮单个模型搜索时间超过 13h, 无结果	最长不可能差分 13 轮, 超过 13 轮的所有模型均有解

5.4 积分分析

积分分析可以看出是一种代数攻击, 其主要寻求关于若干输入明文比特的零和区分器. 可分性 (division property) 是目前分析对称密码算法积分性质最有力的工具^[24, 25], 其同样可以借助 MILP 的方法方便快速地搜索算法的积分区分器. 借助可分性可以搜索得到 FBC128-128 和 FBC128-256 算法的 7 轮积分区分器, 受限计算能力, 目前还无法断定 FBC128-128 和 FBC128-256 是否存在 8 轮积分区分器. FBC256-256 可以搜索得到 10 轮积分区分器, 但是目前同样无法断 FBC256-256 是否存在 11 轮积分区分器.

5.5 零相关分析

零相关分析是线性分析的一种变体^[26], 也是一种常见的分组密码分析方法. 类似于不可能差分分析, 我们利用 MILP 方法搜索 FBC 算法族零相关的最长轮数. 构造模型的方法同不可能差分分析的 M3, 区别在于对轮函数线性层描述与差分分析相反, 但实验结果同不可能差分结果一样, 对于 FBC128-128 和 FBC128-256 的零相关分析, 目前能找到的最大轮数为 7. 对于 FBC256-256 的零相关分析, 目前能找到的最大轮数为 13.

6 软硬件性能评估

6.1 软件性能评估

FBC 算法具有灵活高效的多种软件实现方式, 包括快速无查表实现、4 进 w 出查表实现、8 进 w 出查表实现, 在内存允许的条件下, 甚至可以做成 16 进 w 出的查大表实现. 其中查表实现方法类似于 AES 算法的查表实现, 只是对 FBC 而言, 需要细心调整明密文和密钥比特的位置. 下面我们详细给出了快速无查表实现方法, 具体过程如下:

```
u = x ⊕ k;
u0 = u & bitmask_const;
u1 = (u >> w/4) & bitmask_const;
u2 = (u >> w/2) & bitmask_const;
u3 = (u >> (3 × w/4));
v0 = u0 ⊕ bitmask_const;
u0 = u2 ⊕ bitmask_const;
v0 = v0 ⊕ (u3 & u0);
v1 = u1 ⊕ bitmask_const;
u1 = u3 ⊕ bitmask_const;
v1 = v1 ⊕ (v0 & u1);
v2 = u0 ⊕ (v1 & (v0 ⊕ bitmask_const));
v3 = u1 ⊕ (v2 & (v1 ⊕ bitmask_const));
```

$$y = v0|(v1 \ll w/4)|(v2 \ll w/2)|(v3 \ll (3 \times w/4))$$

注: 这里 bitmask_const 为比特掩码常量, 其为宽度为 $w/4$ 的全 1 字符串.

从上述快速实现方式中, 可以看出, 函数 F 可以通过 7 个异或, 7 个逻辑与运算, 6 个非运算 (与全 1 常量异或运算可以看成是非运算), 6 个移位运算, 3 个或运算共 29 个基础运算就可以实现.

因此 FBC 算法的一轮迭代只需要 $29 + 4 = 33$ 个基础运算便可以实现. 整个加密过程和解密过程只需要 $33r$ 个基础运算就可以完成.

6.1.1 软件测试结果

在 64 位 Windows 以及 32 位 ARM 环境下, 对 FBC 算法的加解密速率进行了测试, 测试环境见表 7, 结果取 10^5 次 CBC 模式运算测试结果的平均值为加/解密速率, 如表 8 所示.

表 7 FBC 算法软件测试环境
Table 7 Software test environment of FBC

测试对象	测试环境
64 bit Windows 环境下的算法速率优化 C 实现	i7-4700 CPU 3.40GHz 主频, 64 位操作系统 Win10, 8 GB 内存, X86-64 环境 编译器: visual studio 2019, x64 release
32 bit ARM 环境下的算法速率优化 C 实现	基于 NXP MIMXRT1060-EVK 主板 (Cortex M7, 600MHz, internal SRAM 1M, 32K data cache, 32K instruction cache) 编译器: arm-none-eabi-gcc 7.3.1 20180622 release

表 8 FBC 算法软件测试结果
Table 8 Software implementation rate of FBC

算法软件实现类别		加密速率 (Mbps)	解密速率 (Mbps)
64 bit Windows 环境下的算法 速率优化 C 实现	128/128	293.21	306.61
	128/256	221.72	233.07
	256/256	339.08	350.65
32 bit ARM 环境下的算法 速率优化 C 实现	128/128	38.57	38.66
	128/256	29.24	27.95
	256/256	22.66	21.78

6.2 硬件性能评估

6.2.1 硬件实现概述

FBC 算法所有版本均可以轻量化实现. 正如 3.2.1 节所示, S 盒可以采用 NFSR 方式实现, 此时需要 4 个与门, 4 个异或门, 5 个非门. 而对于线性变换 L , 每比特需要 2 个异或门. 因此实现函数 F , 需要 w 个与门, $3w$ 个异或门和 $5w/4$ 个非门.

对于 FBC 算法的轮函数可以采用多种方式实现. 一种是同时实现 4 路两重 Feistel 结构, 此时需要 $2w$ 个与门, $10w$ 个异或门和 $5w/2$ 个非门. 另外一种是实现 2 路 Feistel 结构, 另外 2 路采用重复调用即可. 此时需要 w 个与门, $5w$ 个异或门和 $5w/4$ 个非门就可以了. 当然在 Feistel 结构内部, 我们还可以更小宽度来实现轮函数, 此时需要的硬件门数更少.

从上面的讨论可知, FBC 算法的硬件实现代价非常低.

6.2.2 Verilog 仿真实验测试结果

对 FBC 算法进行了 Verilog 硬件仿真实验测试, 测试环境为基于 Xilinx 公司的 ISE Design Suite 14.7 软件工具和 Xilinx virtex 5 FPGA 芯片. 结果如表 9 所示.

表 9 FBC 算法 Verilog 硬件仿真测试
Table 9 Verilog hardware simulation test of FBC

	自测试结果		
	128/128	128/256	256/256
运算周期	$(48 + 1) \times 32 = 1568$	$(64 + 1) \times 32 = 2080$	$(80 + 1) \times 32 = 2592$
时钟约束	5 ns	5 ns	5 ns
加密速率	522 Mbps	394 Mbps	632 Mbps
解密速率	522 Mbps	394 Mbps	632 Mbps
面积	12 696 LUTs	15 400 LUTs	26 486 LUTs
加密吞面比	0.92%	0.75%	21.9%
解密吞面比	1.28%	1.06%	17.8%

致谢

本文作者衷心地感谢武汉大学唐明教授团队对 FBC 算法的硬件评估进行了 Verilog 仿真实现, 并给出了相应结果.

References

[1] DIFFIE W, LEDIN G. SMS4 Encryption Algorithm for Wireless Networks[J]. IACR Cryptology ePrint Archive, 2008: 2008/329.

[2] SCHNEIER B, KELSEY J. Unbalanced Feistel networks and block cipher design[C]. In: Fast Software Encryption—FSE 1996. Springer Berlin Heidelberg, 1996: 121–144. [DOI: 10.1007/3-540-60865-6_49]

[3] HONG S, LEE S, LIM J, et al. Provable security against differential and linear cryptanalysis for the SPN structure[C]. In: Fast Software Encryption—FSE 2000. Springer Berlin Heidelberg, 2000: 273–283. [DOI: 10.1007/3-540-44706-7_19]

[4] BIHAM E. A fast new DES implementation in software[C]. In: Fast Software Encryption—FSE 1997. Springer Berlin Heidelberg, 1997: 260–272. [DOI: 10.1007/BFb0052352]

[5] BOGDANOV A, KNUDSEN L R, LEANDER G, et al. PRESENT: An ultra-lightweight block cipher[C]. In: Cryptographic Hardware and Embedded Systems—CHES 2007. Springer Berlin Heidelberg, 2007: 450–466. [DOI: 10.1007/978-3-540-74735-2_31]

[6] BIHAM E, ANDERSON R, KNUDSEN L. Serpent: A new block cipher proposal[C]. In: Fast Software Encryption—FSE 1998. Springer Berlin Heidelberg, 1998: 222–238. [DOI: 10.1007/3-540-69710-1_15]

[7] ZHANG W, BAO Z, LIN D, et al. RECTANGLE: A bit-slice lightweight block cipher suitable for multiple platforms[J]. SCIENCE CHINA Information Sciences, 2015, 58(12): 1–15. [DOI: 10.1007/s11432-015-5459-7]

[8] BERTONI G, DAEMEN J, PEETERS M, et al. The Keccak SHA-3 submission[OL]. <https://keccak.team/files/Keccak-submission-3.pdf>. January 2011.

[9] MOUHA N, WANG Q J, GU D W, et al. Differential and linear cryptanalysis using mixed-integer linear programming[C]. In: Information Security and Cryptology—INSCRYPT 2011. Springer Berlin Heidelberg, 2011: 57–76. [DOI: 10.1007/978-3-642-34704-7_5]

[10] SUN S W, HU L, WANG P, et al. Automatic security evaluation and (related-key) differential characteristic search: Application to SIMON, PRESENT, LBlock, DES(L) and other bit-oriented block ciphers[C]. In: Advances in Cryptology—ASIACRYPT 2014, Part I. Springer Berlin Heidelberg, 2014: 158–178. [DOI: 10.1007/978-3-662-45611-8_9]

[11] FU K, WANG M Q, GUO Y H, et al. MILP-based automatic search algorithms for differential and linear trails for Speck[C]. In: Fast Software Encryption—FSE 2016. Springer Berlin Heidelberg, 2016: 268–288. [DOI: 10.1007/978-3-662-52993-5_14]

[12] BIHAM E, SHAMIR A. Differential cryptanalysis of DES-like cryptosystems[J]. Journal of Cryptography, 1991, 4(1): 3–72. [DOI: 10.1007/BF00630563]

[13] BIHAM E, SHAMIR A. Differential Cryptanalysis of Snefru, Khafre, REDOC-II, LOKI and Lucifer[C]. In: Advances in Cryptology—CRYPTO '91. Springer Berlin Heidelberg, 1992: 156–171. [DOI: 10.1007/3-540-46766-

- 1_11]
- [14] BIHAM E, SHAMIR A. Differential cryptanalysis of Feal and N-hash[C]. In: Advances in Cryptology—EUROCRYPT '91. Springer Berlin Heidelberg, 1991: 1–16. [DOI: 10.1007/3-540-46416-6_1]
 - [15] BIHAM E, SHAMIR A. Differential cryptanalysis of the full 16-round DES[C]. In: Advances in Cryptology—CRYPTO '92. Springer Berlin Heidelberg, 1993: 487–496. [DOI: 10.1007/3-540-48071-4_34]
 - [16] MATSUI M. Linear cryptanalysis method for DES cipher[C]. In: Advances in Cryptology—EUROCRYPT '93. Springer Berlin Heidelberg, 1994: 386–397. [DOI: 10.1007/3-540-48285-7_33]
 - [17] KALISKIJR B S, ROBSHAW M J B. Linear cryptanalysis using multiple approximations[C]. In: Advances in Cryptology—CRYPTO '94. Springer Berlin Heidelberg, 1994: 26–39. [DOI: 10.1007/3-540-48658-5_4]
 - [18] Baignères T, JUNOD P, VAUDENAY S. How far can we go beyond linear cryptanalysis?[C]. In: Advances in Cryptology—ASIACRYPT 2004. Springer Berlin Heidelberg, 2004: 432–450. [DOI: 10.1007/978-3-540-30539-2_31]
 - [19] CHO J Y. Linear cryptanalysis of reduced-round PRESENT[C]. In: Topics in Cryptology—CT-RSA 2010. Springer Berlin Heidelberg, 2010: 302–317. [DOI: 10.1007/978-3-642-11925-5_21]
 - [20] SEKI H, KANEKO T. Cryptanalysis of five rounds of CRYPTON using impossible differentials[C]. In: Advances in Cryptology—ASIACRYPT 1999. Springer Berlin Heidelberg, 1999: 43–51. [DOI: 10.1007/978-3-540-48000-6_5]
 - [21] BIHAM E, KELLER N. Cryptanalysis of reduced variants of Rijndael[C/OL]. In: Proceedings of the 3rd AES Candidate Conference, 2000. <http://madchat.fr/crypto/codebreakers/35-ebiham.pdf>
 - [22] CHEON J H, KIM M, KIM K, et al. Improved impossible differential cryptanalysis of Rijndael and Crypton[C]. In: Information Security and Cryptology—ICISC 2001. Springer Berlin Heidelberg, 2001: 39–49. [DOI: 10.1007/3-540-45861-1_4]
 - [23] WANG Q, LIU Z, VARICI K, et al. Cryptanalysis of reduced-round SIMON32 and SIMON48[C]. In: Progress in Cryptology—INDOCRYPT 2014. Springer Cham, 2014: 143–160. [DOI: 10.1007/978-3-319-13039-2_9]
 - [24] TODO Y, MORII M. Bit-based division property and application to SIMON family[C]. In: Fast Software Encryption—FSE 2016. Springer Berlin Heidelberg, 2016: 357–377. [DOI: 10.1007/978-3-662-52993-5_18]
 - [25] XIANG Z, ZHANG W, BAO Z, et al. Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers[C]. In: Advances in Cryptology—ASIACRYPT 2016, Part I. Springer Berlin Heidelberg, 2016: 648–678. [DOI: 10.1007/978-3-662-53887-6_24]
 - [26] BOGDANOV A, RIJMEN V. Linear hulls with correlation zero and linear cryptanalysis of block ciphers[J]. Designs, Codes and Cryptography, 2014, 70(3): 369–383. [DOI: 10.1007/s10623-012-9697-z]

作者信息



冯秀涛 (1978–), 湖北孝感人, 博士, 副研究员, 博士生导师. 主要研究领域为对称密码算法设计与分析.
fengxt@amss.ac.cn



曾祥勇 (1973–), 湖北仙桃人, 博士, 教授, 博士生导师. 主要研究领域为密码学.
xzeng@hubu.edu.cn



张凡 (1989–), 湖北孝感人, 博士, 工程师. 主要研究领域为信息安全.
tzhangfan4@163.com



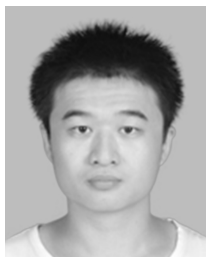
曾光 (1980–), 吉林吉林人, 博士, 副教授, 硕士生导师. 主要研究领域为密码算法分析和密码应用.
49034049@qq.com



唐灯 (1984-), 四川德阳人, 博士, 副教授, 博士生导师. 主要研究方向为密码函数.
dtang@foxmail.com



甘国华 (1978-), 湖北咸宁人, 博士生, 高级工程师. 主要研究领域为软件工程和区块链.
ganguohua01@163.com



王永兴 (1995-), 河北邯郸人, 博士生. 主要研究领域为对称密码.
wangyongxing17@mails.
ucas.edu.cn

附录 A 差分分析活动 S 盒个数不等式刻画求解

下面以 FBC128-128 为例介绍 MILP 具体分析过程.

设 F 函数的输入记作 (x, k) , 这里 k 表示相应的轮密钥. 设 $x = (x_0, x_1, \dots, x_{31}) \in \{0, 1\}^{32}$, 将其写成如下形式:

$$\begin{array}{cccccccc} x_0, & x_1, & x_2, & x_3, & x_4, & x_5, & x_6, & x_7, \\ x_8, & x_9, & x_{10}, & x_{11}, & x_{12}, & x_{13}, & x_{14}, & x_{15}, \\ x_{16}, & x_{17}, & x_{18}, & x_{19}, & x_{20}, & x_{21}, & x_{22}, & x_{23}, \\ x_{24}, & x_{25}, & x_{26}, & x_{27}, & x_{28}, & x_{29}, & x_{30}, & x_{31} \end{array}$$

注意到线性变换 $L(x) = x \oplus (x \lll 3) \oplus (x \lll 10)$, x 的每一列经过 L 变换后仍然在同一列中. 引入二元变量组 $(a_0, a_1, \dots, a_7)$, 分别对应 x 的每一列, 即每个 S 盒, a_i 刻画第 i 列的活动 S 盒, 即当该列为零时, 即对应的 S 盒输入没有差分, a_i 取值为 0; 否则取值为 1. 接下来我们对轮函数用线性不等式刻画, 将最少活动 S 盒个数作为目标函数并构造相应的 MILP 问题, 最后用 Gurobi 求解目标值.

(1) 对函数 F 的差分模式的不等式刻画

设 $(a_0, a_1, \dots, a_7)$ 表示 F 函数的输入差分模式, $(b_0, b_1, \dots, b_7)$ 表示 F 函数的输出差分模式. 为了刻画 $L(x) = x \oplus (x \lll 3) \oplus (x \lll 10)$ 变换的输入输出差分模式, 我们穷尽 L 变换的所有输入, 最后我们发现输入输出差分重量之和为 4 的只有下列 8 种.

$$\begin{array}{ll} 1000000010000110 & 0100000001000011 \\ 0010000010100001 & 0001000011010000 \\ 0000100001101000 & 0000010000110100 \\ 0000001000011010 & 0000000100001101 \end{array}$$

而其余情况其重量或者为零或者大于等于 6. 于是我们用线性不等式刻画如下:

$$\begin{array}{l} a_0 + a_1 + a_2 + a_3 + a_4 + a_5 + a_6 + a_7 = d_0 \\ b_0 + b_1 + b_2 + b_3 + b_4 + b_5 + b_6 + b_7 = 3d_0 \end{array}$$

$$\begin{aligned}
a_0 + a_1 + \cdots + a_7 + b_0 + b_1 + \cdots + b_7 &\geq 6d_1 \\
d_0 + d_1 &= d_2 \\
a_0 + a_1 + a_2 + a_3 + a_4 + a_5 + a_6 + a_7 &\geq d_2 \\
b_0 + b_1 + b_2 + b_3 + b_4 + b_5 + b_6 + b_7 &\geq d_2 \\
d_2 &\geq a_0 \\
d_2 &\geq a_1 \\
&\vdots \\
d_2 &\geq a_7 \\
d_2 &\geq b_0 \\
d_2 &\geq b_1 \\
&\vdots \\
d_2 &\geq b_7
\end{aligned}$$

这里 d_0, d_1 和 d_2 为引入的辅助变量, 这三组变量的取值均为 $\{0,1\}$.

(2) 对轮函数的差分模式的不等式刻画

设第 i 轮函数的输入记作 $(in_0^{(i)}, in_1^{(i)}, in_2^{(i)}, in_3^{(i)})$, 输出记作 $(out_0^{(i)}, out_1^{(i)}, out_2^{(i)}, out_3^{(i)})$, 则有

$$\begin{aligned}
out_0^{(i)} &= F(in_0^{(i)}) \oplus in_1^{(i)}, \quad out_1^{(i)} = in_0^{(i)} \oplus out_3^{(i)}, \\
out_2^{(i)} &= out_0^{(i)} \oplus in_3^{(i)}, \quad out_3^{(i)} = F(in_3^{(i)}) \oplus in_2^{(i)}
\end{aligned}$$

不妨设第 i 轮的输入差分模式记作 $(v_0^{(i)}, v_1^{(i)}, \cdots, v_{31}^{(i)})$, 输出差分模式记作 $(v_0^{(i+1)}, v_1^{(i+1)}, \cdots, v_{31}^{(i+1)})$, 即第 $i+1$ 轮的输入差分模式, 则 $(v_0^{(i)}, v_1^{(i)}, \cdots, v_7^{(i)}), (v_8^{(i)}, v_9^{(i)}, \cdots, v_{15}^{(i)}), (v_{16}^{(i)}, v_{17}^{(i)}, \cdots, v_{23}^{(i)})$ 和 $(v_{24}^{(i)}, v_{25}^{(i)}, \cdots, v_{31}^{(i)})$ 分别对应四个输入 $in_0^{(i)}, in_1^{(i)}, in_2^{(i)}$ 和 $in_3^{(i)}$ 的差分模式, 而 $(v_0^{(i+1)}, v_1^{(i+1)}, \cdots, v_7^{(i+1)}), (v_8^{(i+1)}, v_9^{(i+1)}, \cdots, v_{15}^{(i+1)}), (v_{16}^{(i+1)}, v_{17}^{(i+1)}, \cdots, v_{23}^{(i+1)})$ 和 $(v_{24}^{(i+1)}, v_{25}^{(i+1)}, \cdots, v_{31}^{(i+1)})$ 分别对应四个输出 $out_0^{(i)}, out_1^{(i)}, out_2^{(i)}$ 和 $out_3^{(i)}$ 的差分模式.

引入中间变量 $(m_0^{(i)}, m_1^{(i)}, \cdots, m_{15}^{(i)})$, 即两个 F 函数的输出差分模式, 则根据 F 函数的不等式刻画需要对 $\{(v_0^{(i)}, v_1^{(i)}, \cdots, v_7^{(i)}), (m_0^{(i)}, m_1^{(i)}, \cdots, m_7^{(i)})\}$ 和 $\{(v_{24}^{(i)}, v_{25}^{(i)}, \cdots, v_{31}^{(i)}), (m_8^{(i)}, m_9^{(i)}, \cdots, m_{15}^{(i)})\}$ 这两对输入输出差分模式做相应的不等式刻画. 对于 $\{(m_0^{(i)}, m_1^{(i)}, \cdots, m_7^{(i)}), (v_8^{(i)}, v_9^{(i)}, \cdots, v_{15}^{(i)}), (v_0^{(i+1)}, v_1^{(i+1)}, \cdots, v_7^{(i+1)})\}$ 需要做出如下的不等式刻画:

$$\begin{aligned}
m_j^{(i)} + v_{j+8}^{(i)} + v_j^{(i+1)} &\geq 2d_j^{(i)} \\
d_j^{(i)} &\geq m_j^{(i)} \\
d_j^{(i)} &\geq v_{j+8}^{(i)} \\
d_j^{(i)} &\geq v_j^{(i+1)}
\end{aligned}$$

这里 $j = 0, 1, \cdots, 7$, $d_j^{(i)}$ 为辅助变量, 取值为 $\{0,1\}$.

同理需要

$$\begin{aligned}
&\{(v_8^{(i+1)}, v_9^{(i+1)}, \cdots, v_{15}^{(i+1)}), (v_0^{(i)}, v_1^{(i)}, \cdots, v_7^{(i)}), (v_{24}^{(i+1)}, v_{25}^{(i+1)}, \cdots, v_{31}^{(i+1)})\} \\
&\{(v_{16}^{(i+1)}, v_{17}^{(i+1)}, \cdots, v_{23}^{(i+1)}), (v_0^{(i+1)}, v_1^{(i+1)}, \cdots, v_7^{(i+1)}), (v_{24}^{(i)}, v_{25}^{(i)}, \cdots, v_{31}^{(i)})\} \\
&\{(v_{24}^{(i+1)}, v_{25}^{(i+1)}, \cdots, v_{31}^{(i+1)}), (m_8^{(i)}, m_9^{(i)}, \cdots, m_{15}^{(i)}), (v_{16}^{(i)}, v_{17}^{(i)}, \cdots, v_{23}^{(i)})\}
\end{aligned}$$

做相应的不等式刻画.

综上每一轮引入 16 个临时变量 $(m_0^{(i)}, m_1^{(i)}, \dots, m_{15}^{(i)})$ 以及 38 个辅助变量.

(3) 限制条件刻画

首先, 设初始输入差分模式记作 $(v_0^{(0)}, v_1^{(0)}, \dots, v_{31}^{(0)})$, 则有

$$v_0^{(0)} + v_1^{(0)} + \dots + v_{31}^{(0)} \geq 1$$

该不等式表示输入差分不为零.

其次, 所有变量取值范围均为 $\{0, 1\}$.

(4) 目标函数刻画

对于每一轮的输入差分模式 $(v_0^{(i)}, v_1^{(i)}, \dots, v_{31}^{(i)})$, 活跃 S 盒的体现在 $(v_0^{(i)}, v_1^{(i)}, \dots, v_7^{(i)})$ 和 $(v_{24}^{(i)}, v_{25}^{(i)}, \dots, v_{31}^{(i)})$, 即两个 F 函数的输入差分模式. 因此目标函数为

$$\min \sum_{i=0}^{n-1} (v_0^{(i)} + v_1^{(i)} + \dots + v_7^{(i)} + v_{24}^{(i)} + v_{25}^{(i)} + \dots + v_{31}^{(i)})$$

这里 n 表示轮数.

我们利用 Gurobi 运行该 MILP 问题, 当轮数为 32 时, 活动 S 盒的个数为 64; 当轮数为 64 时, 活动 S 盒的个数为 128.

附录 B 线性分析活动 S 盒个数不等式刻画求解

下面以 FBC128-128 为例给出了其 MILP 分析过程.

设 $x = (x_0, x_1, \dots, x_{31}) \in \{0, 1\}^{32}$ 表示 F 函数的输入, 引入二元变量组 $(a_0, a_1, \dots, x_7)$, 分别对应 x 的每一列, 即每个 S 盒. a_i 画第 i 列的活动 S 盒, 即当该列为零时, 即对应 S 盒的输入线性掩码为 0, a_i 取值为 0; 否则取值为 1. 然后我们对轮函数用线性不等式刻画, 将最少活动 S 盒个数作为目标函数并构造相应的 MILP 问题, 最后用 Gurobi 求解目标值.

(1) 对 F 函数的线性掩码模式的不等式刻画

设 $(a_0, a_1, \dots, x_7)$ 表示 F 函数的输入线性掩码模式, $(b_0, b_1, \dots, b_7)$ 表示 F 函数的输出线性掩码模式. 根据线性掩码与差分的对偶性, 此时我们穷尽线性变换 $L^T(x) = x \oplus (x \lll 3) \oplus (x \lll 10)$ (该线性变换为 $L(x)$ 的转置) 的所有输入, 最后我们发现输入输出线性掩码重量为 4 的只有下列 8 种可能.

$$\begin{array}{ll} 1000000010000110 & 0100000000001101 \\ 0010000000011010 & 0001000000110100 \\ 0000100001101000 & 0000010011010000 \\ 0000001010100001 & 0000000101000011 \end{array}$$

而其余情况其重量或者为零或者大于等于 6. 我们用线性不等式刻画如下:

$$\begin{array}{l} a_0 + a_1 + a_2 + a_3 + a_4 + a_5 + a_6 + a_7 = d_0 \\ b_0 + b_1 + b_2 + b_3 + b_4 + b_5 + b_6 + b_7 = 3d_0 \\ a_0 + a_1 + \dots + a_7 + b_0 + b_1 + \dots + b_7 \geq 6d_1 \\ d_0 + d_1 = d_2 \\ a_0 + a_1 + a_2 + a_3 + a_4 + a_5 + a_6 + a_7 \geq d_2 \\ b_0 + b_1 + b_2 + b_3 + b_4 + b_5 + b_6 + b_7 \geq d_2 \end{array}$$

$$d_2 \geq a_0$$

$$d_2 \geq a_1$$

$$\vdots$$

$$d_2 \geq a_7$$

$$d_2 \geq b_0$$

$$d_2 \geq b_1$$

$$\vdots$$

$$d_2 \geq b_7$$

这里 d_0, d_1 和 d_2 为引入的辅助变量, 这三组变量的取值均为 $\{0,1\}$.

(2) 对轮函数的线性掩码模式的不等式刻画

设第 i 轮函数的输入记作 $(in_0^{(i)}, in_1^{(i)}, in_2^{(i)}, in_3^{(i)})$, 输出记作 $(out_0^{(i)}, out_1^{(i)}, out_2^{(i)}, out_3^{(i)})$, 则有

$$\begin{aligned} out_0^{(i)} &= F(in_0^{(i)}) \oplus in_1^{(i)}, & out_1^{(i)} &= in_0^{(i)} \oplus out_3^{(i)}, \\ out_2^{(i)} &= out_0^{(i)} \oplus in_3^{(i)}, & out_3^{(i)} &= F(in_3^{(i)}) \oplus in_2^{(i)} \end{aligned}$$

不妨设第 i 轮的输入线性掩码模式记作 $(v_0^{(i)}, v_1^{(i)}, \dots, v_{31}^{(i)})$, 输出线性掩码模式记作 $(v_0^{(i+1)}, v_1^{(i+1)}, \dots, v_{31}^{(i+1)})$, 即第 $i+1$ 轮的输入线性掩码模式, 引入中间变量 $(m_0^{(i)}, m_1^{(i)}, \dots, m_{15}^{(i)})$, 即两个 F 函数的输出线性掩码模式, 则根据 F 函数的不等式刻画需要对 $\{(v_8^{(i)}, v_9^{(i)}, \dots, v_{15}^{(i)}), (m_0^{(i)}, m_1^{(i)}, \dots, m_7^{(i)})\}$ 和 $\{(v_{16}^{(i)}, v_{17}^{(i)}, \dots, v_{23}^{(i)}), (m_8^{(i)}, m_9^{(i)}, \dots, m_{15}^{(i)})\}$ 这两对输入输出线性掩码模式做相应的不等式刻画.

对于线性掩码模式组 $\{(v_8^{(i)}, v_9^{(i)}, \dots, v_{15}^{(i)}), (v_{16}^{(i+1)}, v_{17}^{(i+1)}, \dots, v_{23}^{(i+1)}), (v_0^{(i+1)}, v_1^{(i+1)}, \dots, v_7^{(i+1)})\}$, 需要做出如下的不等式刻画:

$$\begin{aligned} v_{j+8}^{(i)} + v_{j+16}^{(i+1)} + v_j^{(i+1)} &\geq 2d_j^{(i)} \\ d_j^{(i)} &\geq v_{j+8}^{(i)} \\ d_j^{(i)} &\geq v_{j+16}^{(i+1)} \\ d_j^{(i)} &\geq v_j^{(i+1)} \end{aligned}$$

这里 $j = 0, 1, \dots, 7$, $d_j^{(i)}$ 为辅助变量, 取值为 $\{0,1\}$.

同理需要对

$$\begin{aligned} &\{(m_0^{(i)}, m_1^{(i)}, \dots, m_7^{(i)}), (v_0^{(i)}, v_1^{(i)}, \dots, v_7^{(i)}), (v_8^{(i+1)}, v_9^{(i+1)}, \dots, v_{15}^{(i+1)})\} \\ &\{(m_8^{(i)}, m_9^{(i)}, \dots, m_{15}^{(i)}), (v_{24}^{(i)}, v_{25}^{(i)}, \dots, v_{31}^{(i)}), (v_{16}^{(i+1)}, v_{17}^{(i+1)}, \dots, v_{23}^{(i+1)})\} \\ &\{(v_0^{(i+1)}, v_1^{(i+1)}, \dots, v_7^{(i+1)}), (v_{16}^{(i)}, v_{17}^{(i)}, \dots, v_{23}^{(i)}), (v_{24}^{(i+1)}, v_{25}^{(i+1)}, \dots, v_{31}^{(i+1)})\} \end{aligned}$$

做相应的不等式刻画.

综上每一轮引入 16 个临时变量 $(m_0^{(i)}, m_1^{(i)}, \dots, m_{15}^{(i)})$ 以及 38 个辅助变量.

(3) 限制条件刻画

首先, 设初始输入线性掩码模式记作 $(v_0^{(0)}, v_1^{(0)}, \dots, v_{31}^{(0)})$, 则有

$$v_0^{(0)} + v_1^{(0)} + \dots + v_{31}^{(0)} \geq 1$$

该不等式表示输入线性掩码不为零.

其次, 所有变量取值范围均为 $\{0,1\}$.

(4) 目标函数刻画

对于每一轮的输入线性掩码模式 $(v_0^{(i)}, v_1^{(i)}, \dots, v_{31}^{(i)})$, 活跃 S 盒的体现在 $(v_8^{(i)}, v_9^{(i)}, \dots, v_{15}^{(i)})$ 和 $(v_{16}^{(i)}, v_{17}^{(i)}, \dots, v_{23}^{(i)})$, 即两个 F 函数的输出线性掩码模式. 因此目标函数为

$$\min \sum_{i=0}^{n-1} (v_8^{(i)} + v_9^{(i)} + \dots + v_{15}^{(i)} + v_{16}^{(i)} + v_{17}^{(i)} + \dots + v_{23}^{(i)})$$

这里 n 表示轮数.

事实上, 线性掩码模式的不等式刻画与差分模式的不等式刻画完全等价, 我们利用 Gurobi 运行该 MILP 问题, 当轮数为 32 时, 活动 S 盒的个数为 64; 当轮数为 64 时, 活动 S 盒的个数为 128.